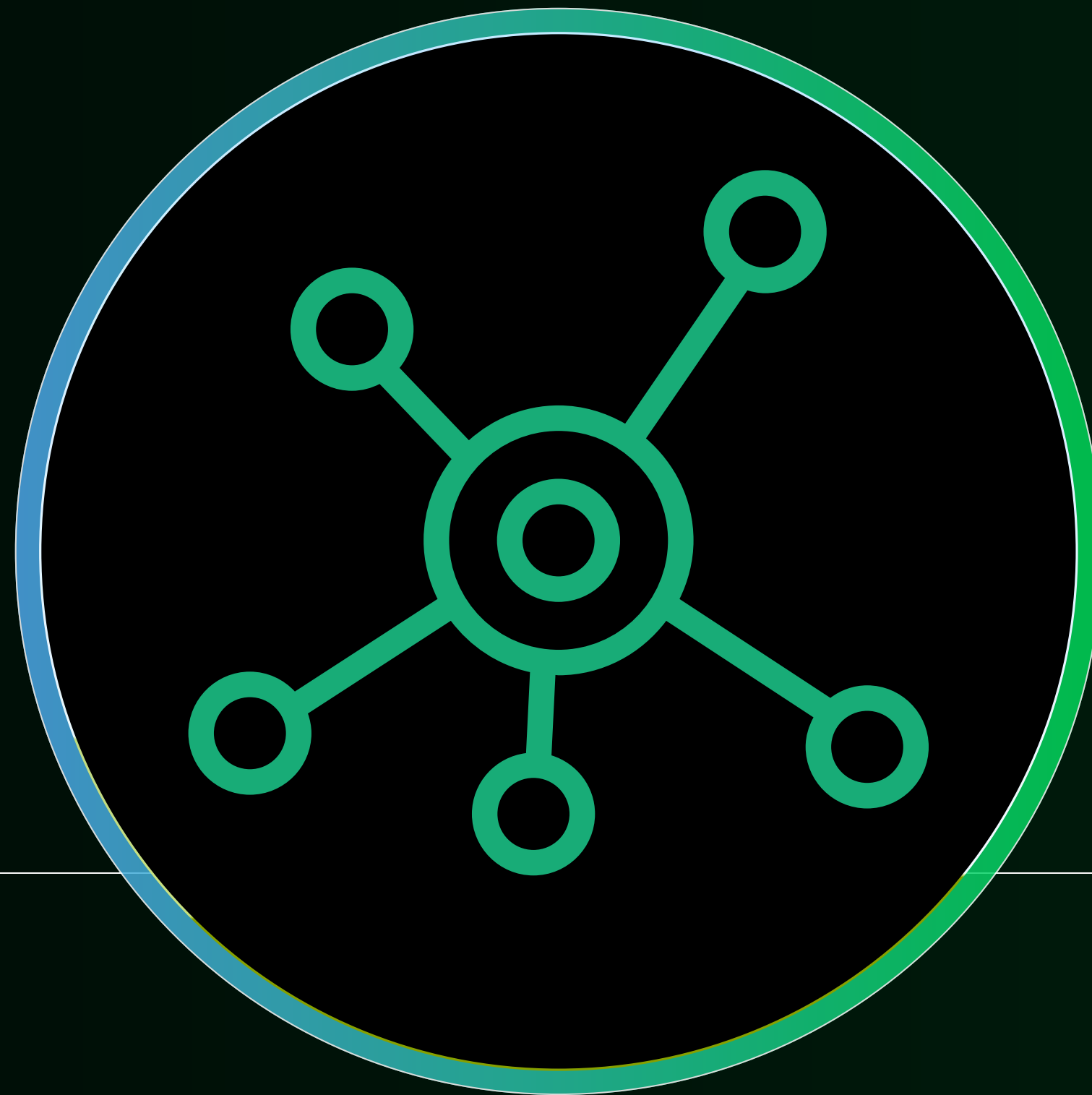
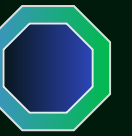




CNN

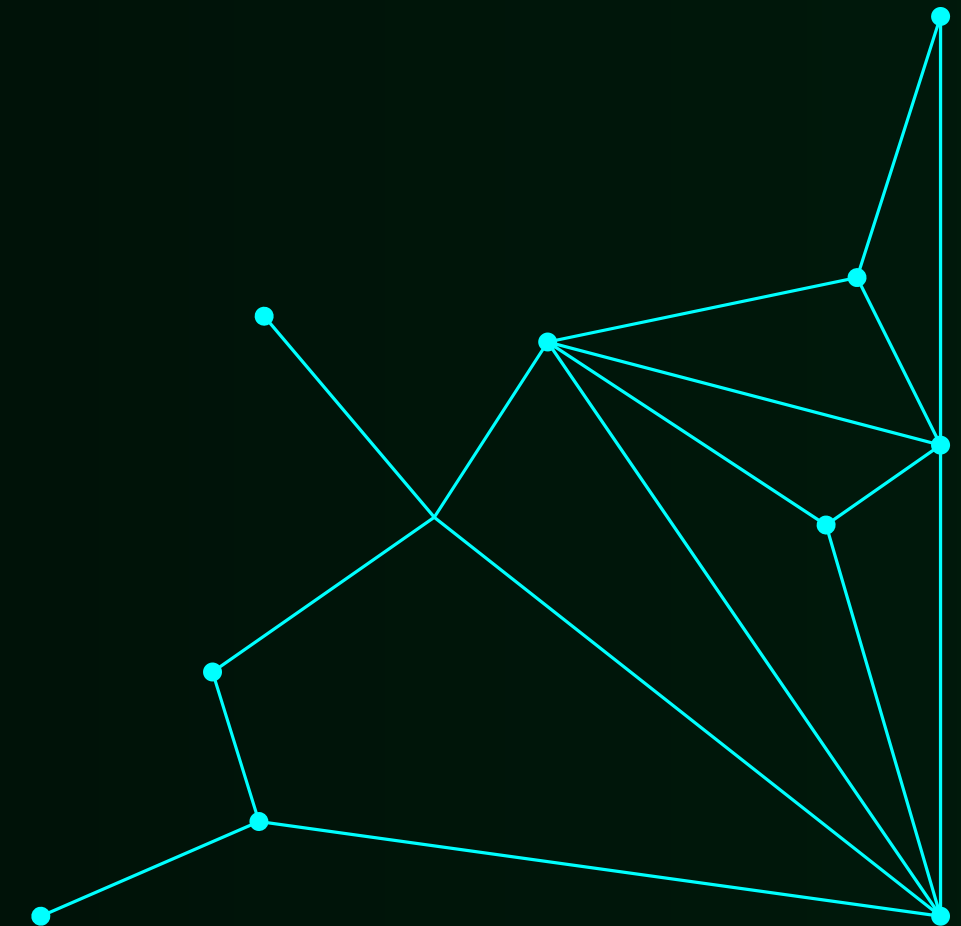
HANDS-ON EXPERIENCE

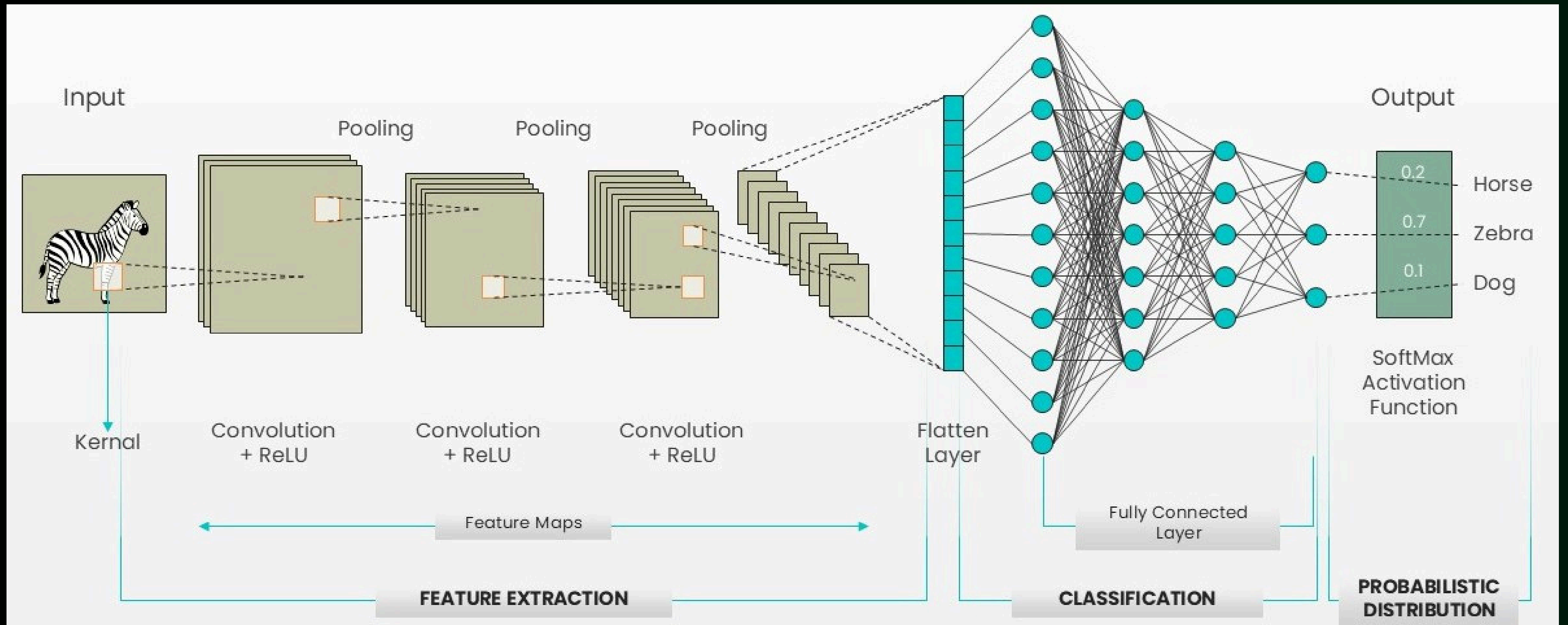
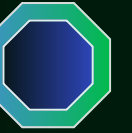


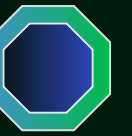


What is a Convolutional Neural Network (CNN)?

- Special type of neural network designed for image and grid-like data.
- Automatically learns to detect patterns: edges, shapes, objects.
- Works like how humans identify objects — from simple (edges) → complex (faces, scenes).
- Applications: image classification, object detection, medical imaging, facial recognition.







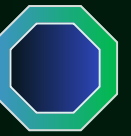
Why Not Regular ANNs?

- Images are high-dimensional (e.g., $256 \times 256 \times 3 = \sim 200k$ inputs).
- Regular ANN would require too many parameters.
- No spatial awareness — treats all pixels as independent.
- CNNs solve this by local connections, parameter sharing, and feature hierarchy.

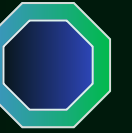




CNN Layers Overview

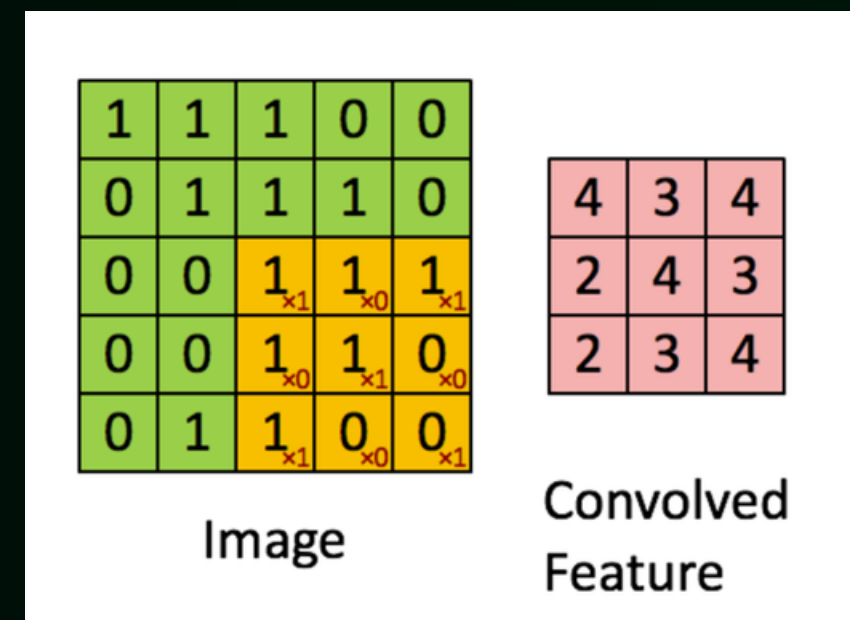
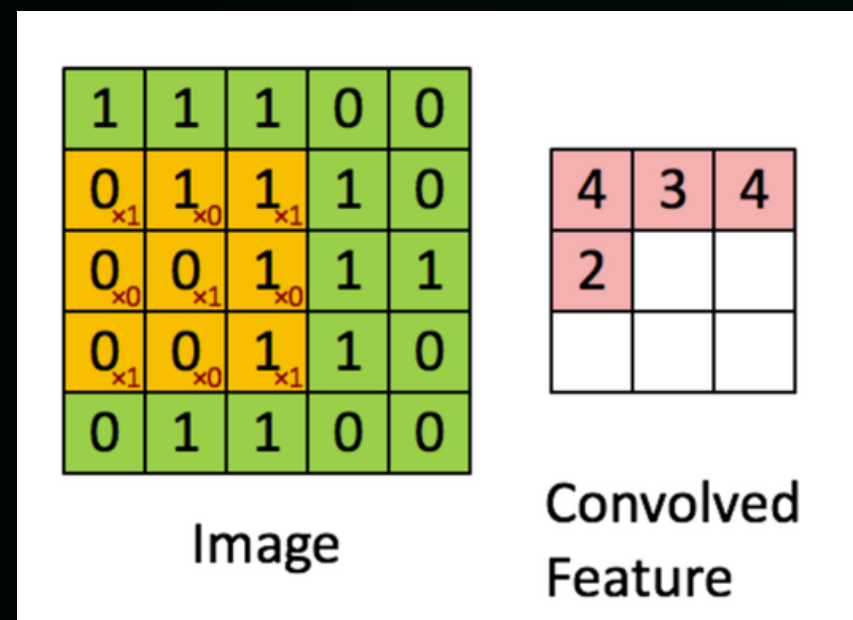
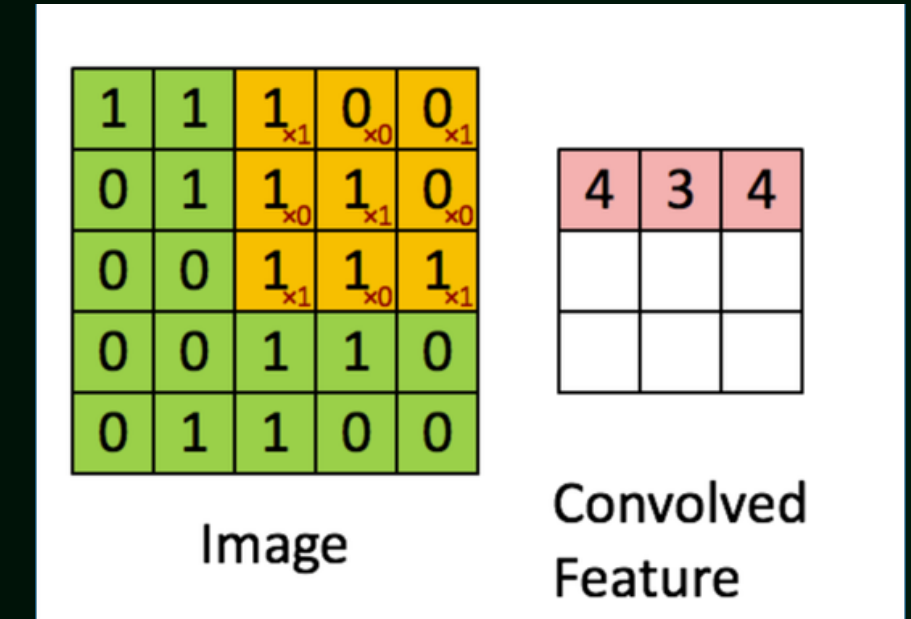
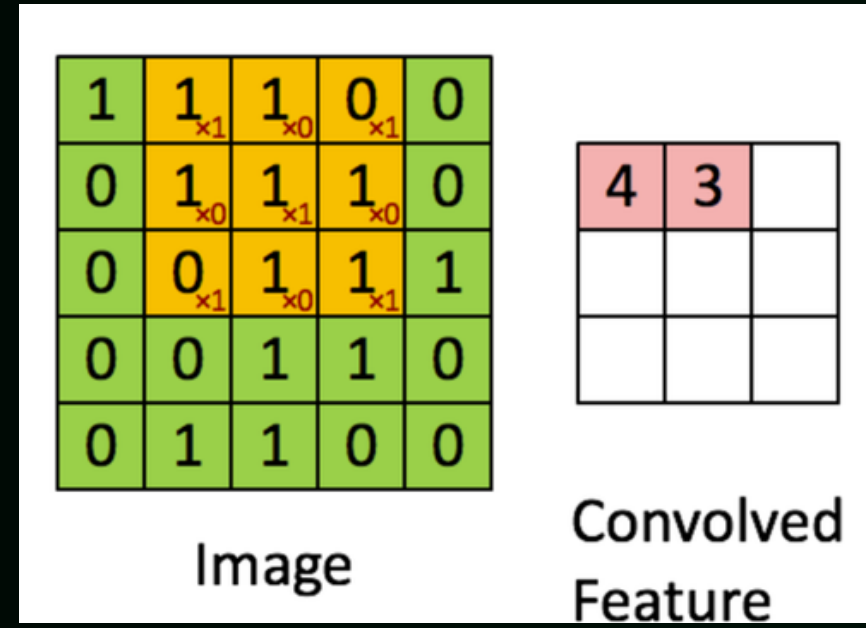
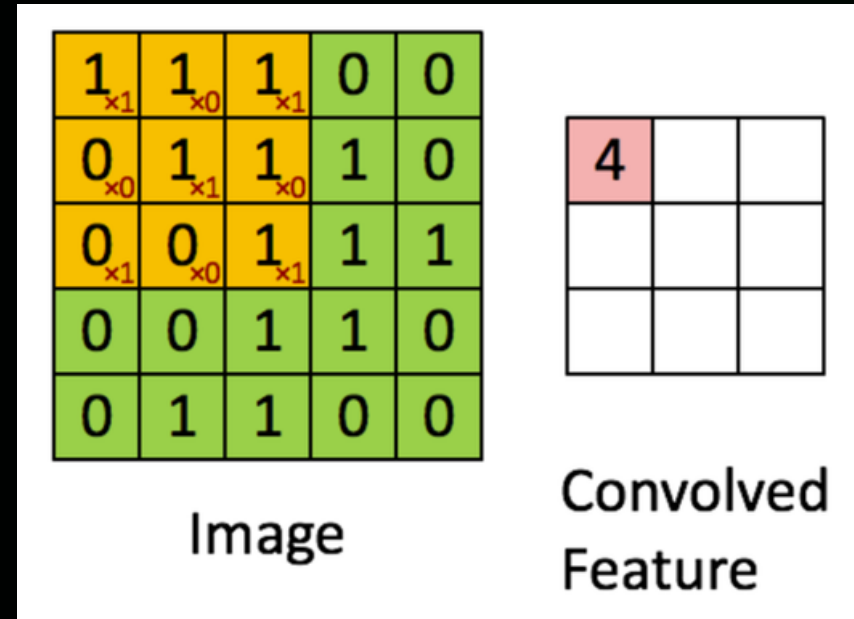


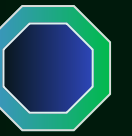
Layer	Purpose
Convolution	Extract local spatial features
Activation (ReLU)	Introduce nonlinearity
Pooling	Reduce dimensions, add invariance
Flatten	Prepare for classification
Fully Connected	Combine features, predict output
Output (Softmax)	Provide class probabilities



Convolutional Layer – Feature Extraction

- Consists of learnable filters (kernels) with small spatial dimensions but full depth (e.g., $a \times a \times 3$)
- Filters slide across the image, performing convolution: dot products over local patches to create feature maps
- Enables parameter sharing and locality, dramatically reducing learnable parameters and emphasizing spatial features





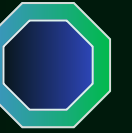
Dimensions after applying a convolution operation:

- Suppose that we have an image whose dimensions are 5x5 and the filter size is 3x3 and stride "S" = 1
- Given the following rule we can calculate the output dimension:

$$\text{Output Size} = \frac{\text{Input Size} - \text{Filter Size} + 1}{\text{Stride}}$$

$$\text{Output Height} = \frac{5 - 3 + 1}{1} = 3$$

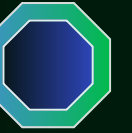
$$\text{Output Width} = \frac{5 - 3 + 1}{1} = 3$$



Padding:

- We can see that the image shrinks with every convolution operation so Padding allows us to control the output size and keep it the same as the input size.
- Another downside of not using Padding is that pixels at the edges of the image are used less frequently in the convolution operation so this will cause “throwing away” a lot of information that is in the edge or the corner.

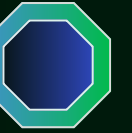




Activation (e.g., ReLU) – Introducing Non-Linearity

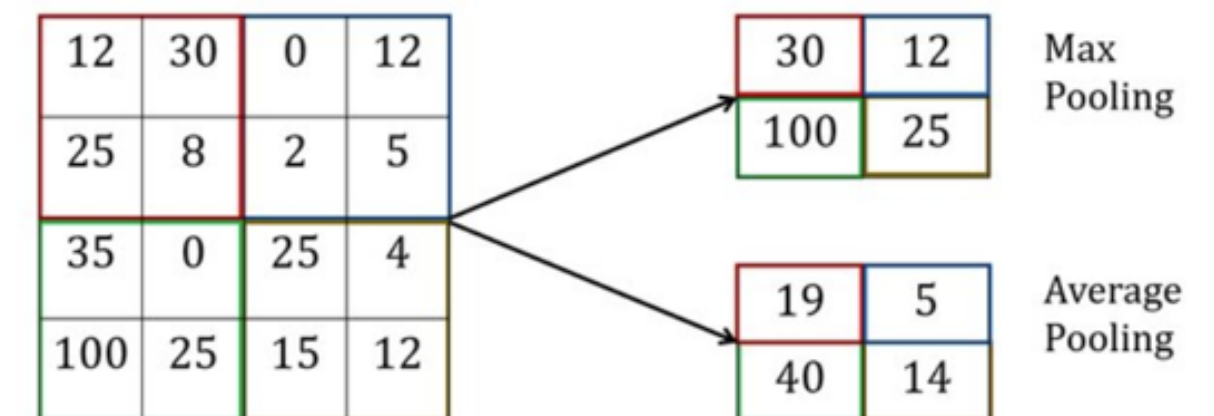
- Applies a non-linear function (commonly ReLU) after convolution
- ReLU zeroes out negatives; keeps positives intact
- Makes the network capable of learning complex, non-linear patterns
- Helps with faster and more effective training.

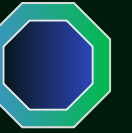
Without non-linear activation, CNNs would just be a sequence of convolutions—incapable of capturing complex relationships. ReLU is simple yet effective and widely used.



Pooling Layer – Dimension Reduction & Invariance

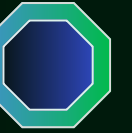
- Downsamples/shrinks feature maps (reduces width/height) using operations like Max Pooling or Average Pooling
- Max Pooling: picks strongest feature.
- Average Pooling: takes average of region.
- Reduces computation and helps control overfitting
- Adds translation invariance, so small shifts in the input don't massively alter feature detection





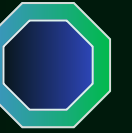
Flatten – Bridge to Dense Layers

- Converts multi-dimensional feature maps into a one-dimensional vector
- Enables passage to fully connected layers, which require flat input



Fully Connected Layer – Final Prediction

- Each neuron connects to all activations from the previous (flattened) layer
- Learns high-level combinations of features to make predictions
- Often ends with an output activation like softmax (for multi-class) or sigmoid (for binary classification)



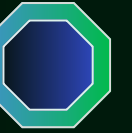
Output Layer – Probability Distribution

- For classification, uses logistic functions: Softmax for multi-class, Sigmoid for binary
- Converts scores into probability outputs that sum to one

Here, scores are converted into probabilities. For example, in a cat vs. dog classifier, you might get [Cat: 0.87, Dog: 0.13].

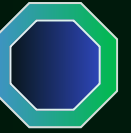
The background features a teal-to-green gradient with several organic, wavy shapes in darker teal, light green, and orange. The text is centered in a bold, white, sans-serif font.

Some CNN architectures



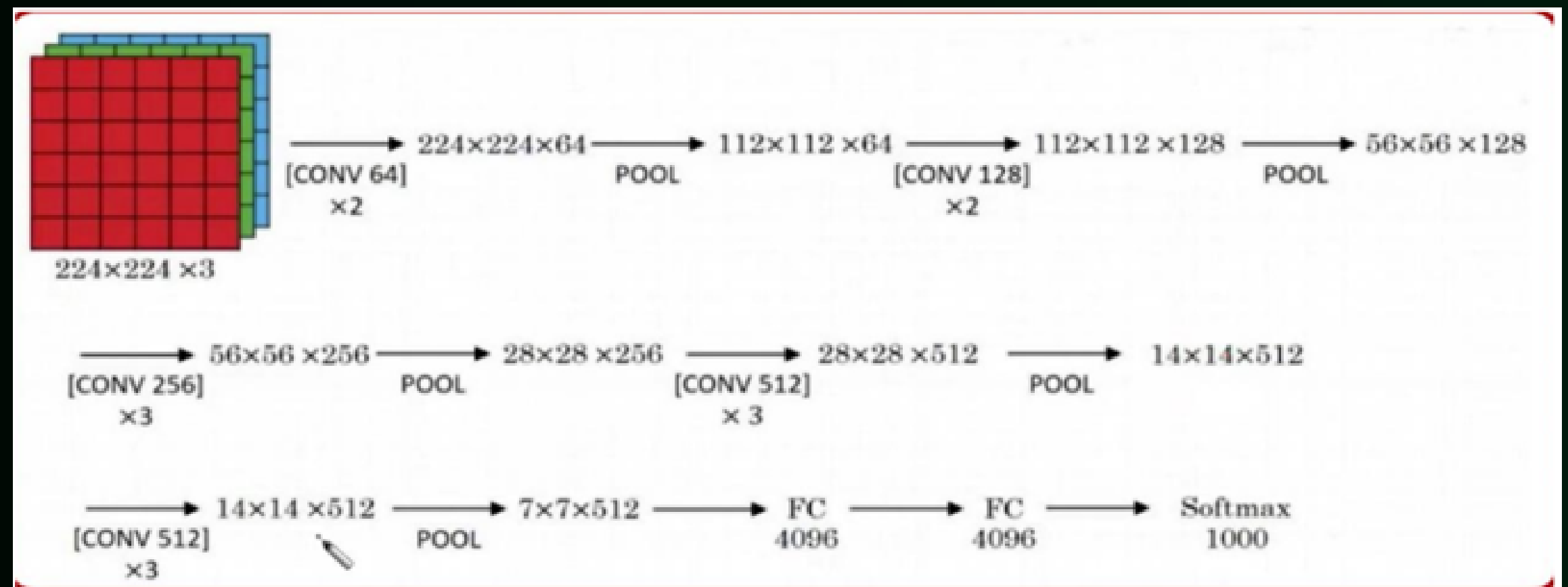
Architectures using CNN:

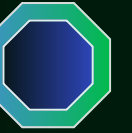
- LeNet-5
- AlexNet
- VGGNet-16
- Resnets
- MobileNet
- EfficientNet



VGGNet-16

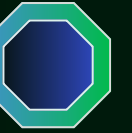
- It follows this arrangement of convolution and max pool layers consistently throughout the whole architecture. In the end it has two fully connected layers, followed by a softmax for output.





ResNet

- However, a huge barrier to training them is vanishing gradients: very deep networks often have a gradient signal that goes to zero quickly, thus making gradient descent prohibitively slow.
- More specifically, during gradient descent, as we backprop from the final layer back to the first layer, we are multiplying by the weight matrix on each step. If the gradients are small, due to large number of multiplications, the gradient can decrease exponentially quickly to zero (or, in rare cases, grow exponentially quickly and “explode” to take very large values)

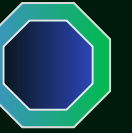


Additional Resources

But what is a convolution?
3Blue1Brown

$P_X * P_Y = P_{X+Y}$

Watch on YouTube



Model	Best Usage	What makes it special	Size/Complexity
VGG-16	Baseline for image classification, transfer learning (small to medium datasets).	Very deep and simple design: just stacked 3×3 convs + max pooling. Easy to understand.	~138M parameters (very large), high memory and compute demand.
ResNet 50	Large-scale image recognition, detection, segmentation. Standard backbone in CV tasks.	Introduced residual connections (skip connections) to train very deep networks without vanishing gradients.	ResNet-50 has ~25M params, deeper versions go larger but still more efficient than VGG.
MobileNet	Mobile/edge deployment, real-time inference on low-power devices.	Uses depthwise separable convolutions → drastically reduces computation while keeping good accuracy.	MobileNetV1 ~4.2M params, MobileNetV2 ~3.4M params (very lightweight).
EfficientNet	High accuracy with good efficiency, SOTA classification, transfer learning.	Compound scaling (depth, width, resolution together) + mobile-friendly building blocks. Balanced trade-off.	EfficientNet-B0 ~5M params, larger versions scale up (B7 ~66M params) but still more efficient than ResNet/VGG.



Q&A



Thank You!